VRAR Report

Rajab Ahsen

vntj44

1 Rendering

The rendering engine was extended to support real-time display, multiple objects, and object transformations. The original implementation wrote frames to disk; this was replaced with real-time display using `cv2.imshow`, enabling continuous visualisation of the framebuffer.

Support for multiple objects was added through a `Model` class storing geometry and transformation parameters, allowing the central bunny, floor, and falling objects to be rendered within the same scene.

Object transformations were implemented using homogeneous matrices:

- Translation to position objects in the scene
- Rotation about the x, y, and z axes
- Scaling to control object size

These are combined into a model matrix and applied prior to projection, enabling correct positioning and animation of all objects. As shown in Figure 1, the engine correctly renders multiple objects with applied transformations.

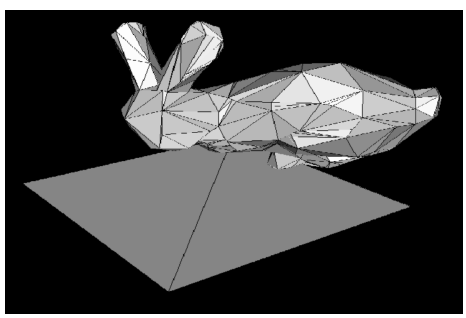

Figure 1: Static render showing multiple objects and transformations.

2 Tracking

2.1 Handling POSE Data

IMU data was imported from the provided CSV file using `numpy.genfromtxt`, extracting timestamps, gyroscope, accelerometer, and magnetometer readings. The gyroscope values were converted from degrees per second to radians per second to ensure consistency with subsequent computations.

Quaternion representations were used to model orientation. The following operations were implemented:

- Conversion from Euler angles to quaternions
- Conversion from quaternions to Euler angles
- Quaternion conjugation for inverse rotation

- Quaternion multiplication for combining rotations

Quaternion multiplication was implemented to combine rotations. Given two quaternions $q_a = (w_a, \mathbf{v}_a)$ and $q_b = (w_b, \mathbf{v}_b)$, the product is:

$$q_a \otimes q_b = (w_a w_b - \mathbf{v}_a \cdot \mathbf{v}_b,\ w_a \mathbf{v}_b + w_b \mathbf{v}_a + \mathbf{v}_a \times \mathbf{v}_b)$$

All quaternions were normalised after each operation:

$$q \leftarrow \frac{q}{\|q\|}$$

2.2 POSE Calculation

For dead reckoning, the initial orientation was set to the identity quaternion $q_0 = [1, 0, 0, 0]$. At each timestep, the gyroscope reading was used to construct a small rotation quaternion from the measured angular velocity and elapsed time. The current orientation was then updated recursively by quaternion multiplication and normalisation. This provides a simple and responsive orientation estimate using only gyroscope data, but any bias or noise in the angular velocity accumulates over time as drift.

To reduce this drift, accelerometer-based tilt correction was added using a complementary filter. After the gyroscope estimate was updated, the accelerometer reading was normalised and rotated into the global frame. This estimated gravity direction was then compared with the global up vector. The tilt axis was computed using the cross product between these two vectors, and the tilt angle was computed from their dot product. A correction quaternion was then formed and applied to the gyroscope estimate, with the correction strength controlled by the complementary filter parameter α.

Additional processing was applied to improve numerical stability. Accelerometer vectors were normalised before use, corrections were skipped when vector magnitudes were too small, the dot product was clipped to the valid range before applying arccos, and very small tilt corrections were ignored. These steps reduce unstable updates and help the fused orientation remain well behaved during playback. As shown in Figure 2, the orientation of the bunny changes between frames captured approximately one second apart, demonstrating that the tracking pipeline produces consistent and continuous motion over time.

Figure 2: Tracked orientation of the bunny at two time steps approximately one second apart using gyroscope dead reckoning with accelerometer tilt correction.

2.2.1 Alpha Value Comparison

The complementary filter parameter α controls the balance between gyroscope integration and accelerometer-based tilt correction. Lower values apply stronger accelerometer correction, while higher values place greater weight on the gyroscope estimate.

Table 1 compares four values of α. Final absolute roll, pitch, and yaw were measured from the final quaternion in the sequence. Smoothness was measured as the mean magnitude of the frame-to-frame quaternion change over the full sequence, with lower values indicating less abrupt orientation updates.

α	Final |Roll|	Final |Pitch|	Final |Yaw|	Smoothness
0.01	0.0806	0.0399	0.1379	0.0124
0.30	0.0820	0.0381	0.1549	0.0084
0.70	0.0806	0.0403	0.1921	0.0042
0.98	0.0725	0.0322	0.2101	0.0017

Table 1: Comparison of complementary filter alpha values for gyroscope and accelerometer fusion.

As α increased, the orientation estimate became significantly smoother, indicating that the accelerometer correction was being applied less aggressively. Lower values such as $\alpha = 0.01$ and $\alpha = 0.30$ produced larger frame-to-frame changes, which would be perceived visually as less stable motion. In this implementation, $\alpha = 0.98$ gave the lowest final roll and pitch errors while also producing the smoothest result, making it the best overall trade-off for the dataset used. Final yaw error remained comparatively large across all values, which is expected because accelerometer fusion mainly corrects tilt and does not directly address yaw drift.

3 Advanced Tracking

To mitigate yaw drift, magnetometer data was incorporated into the complementary filter. The first magnetometer reading was taken as a reference direction and projected onto the horizontal plane defined by the global up vector. At each timestep, the current magnetometer reading was normalised, rotated into the global frame using the current orientation estimate, and projected onto the same plane.

The signed angle between the current projected magnetic direction and the reference direction was then computed about the global up axis. This angle represents the estimated yaw drift. A correction quaternion about the up axis was formed from this error and applied to the current orientation estimate, with the correction strength controlled by a separate yaw filter parameter.

As with the accelerometer correction, additional processing was used to improve robustness. Magnetometer vectors were normalised before use, the horizontal projection was recomputed each frame, and correction was skipped when projected magnitudes became too small. These steps reduce unstable yaw updates and help maintain a more consistent heading estimate over time. As shown in Figure 3, incorporating the magnetometer produces tracked motion using all available sensor inputs.

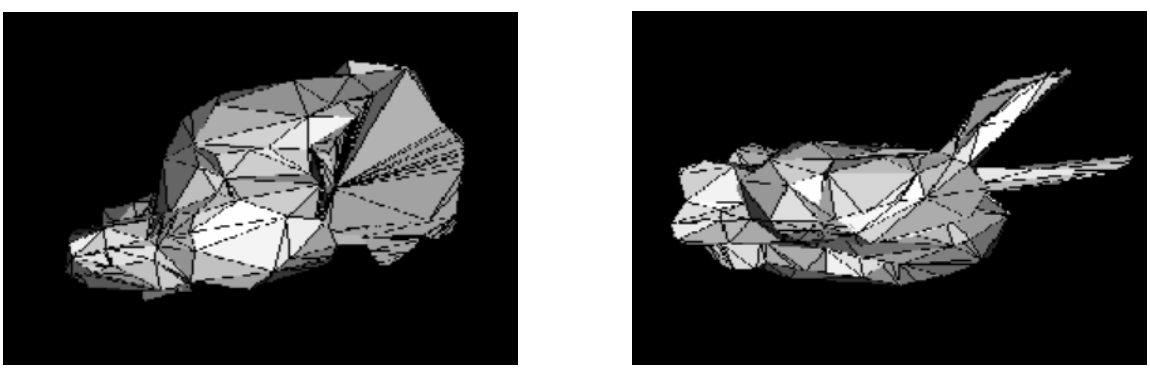

Figure 3: Tracked bunny orientation at two time steps approximately one second apart using gyroscope, accelerometer, and magnetometer fusion.

3.0.1 Alpha Value Comparison

A separate yaw correction parameter was used for magnetometer fusion. Lower values of α_{yaw} apply stronger magnetometer-based heading correction, while higher values place more weight on the current orientation estimate and therefore produce smoother but less aggressive updates.

Table 2 compares four yaw alpha values while keeping the tilt correction fixed at $\alpha_{\text{tilt}} = 0.98$. Final absolute roll, pitch, and yaw were measured from the final quaternion in the sequence.

α_{yaw}	Final |Roll|	Final |Pitch|	Final |Yaw|	Smoothness
0.010	0.0725	0.0322	0.2902	0.0059
0.300	0.0725	0.0322	0.2893	0.0050
0.700	0.0725	0.0322	0.2801	0.0035
0.995	0.0725	0.0322	0.2934	0.0017

Table 2: Comparison of yaw alpha values for gyroscope, accelerometer, and magnetometer fusion.

As expected, increasing α_{yaw} produced smoother motion, with the lowest smoothness value obtained at $\alpha_{\text{yaw}} = 0.995$. Roll and pitch remained unchanged across all four tests because the tilt correction parameter was held constant. The final yaw error varied only modestly, with $\alpha_{\text{yaw}} = 0.700$ giving the lowest final yaw error for this dataset. Qualitatively, lower yaw alpha values produced more aggressive heading corrections, while higher values gave more visually stable motion. In the final implementation, $\alpha_{\text{yaw}} = 0.995$ was retained because it provided the smoothest result while still maintaining comparable heading performance.

4 Physics

Simple physics was implemented for the falling bunny using gravity and air resistance. Gravity was modelled as a constant downward acceleration, while drag was applied in the opposite direction to motion using a quadratic air resistance model

$$\mathbf{F}_d = -\frac{1}{2} C_d \rho A \|\mathbf{v}\| \mathbf{v}.$$

Position and velocity were updated each frame using a forward Euler integration scheme

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{a}\Delta t, \quad \mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}\Delta t.$$

To simulate interaction with the ground efficiently, a simple floor height limit was used rather than performing collision checks against the rendered floor geometry. When the falling bunny reached this vertical limit, its downward velocity was clamped to zero and a constant horizontal deceleration was applied to approximate friction. This provided a lightweight approximation of floor contact.

Distance-based collision detection was implemented using bounding spheres. A collision was detected when

$$\|\mathbf{x}_1 - \mathbf{x}_2\| < r_1 + r_2,$$

where $\mathbf{x}_1, \mathbf{x}_2$ are the object centres and r_1, r_2 their radii. On collision, the falling bunny was moved out of penetration along the collision normal, and its velocity was adjusted using a restitution coefficient e

$$\mathbf{v}' = \mathbf{v} - (1 + e)(\mathbf{v} \cdot \mathbf{n})\mathbf{n}.$$

As shown in Figure 4, the falling bunny collides with the tracked bunny and subsequently interacts with the floor, demonstrating both collision response and force-based motion.

Although the original intention was to have multiple bunnies falling into the tracked bunny, in practice only one falling bunny was used in the final demonstration because simulating and rendering several moving objects at once was too demanding for the laptop used. This still demonstrated both the force-based motion and the required collision behaviour while maintaining acceptable runtime performance.

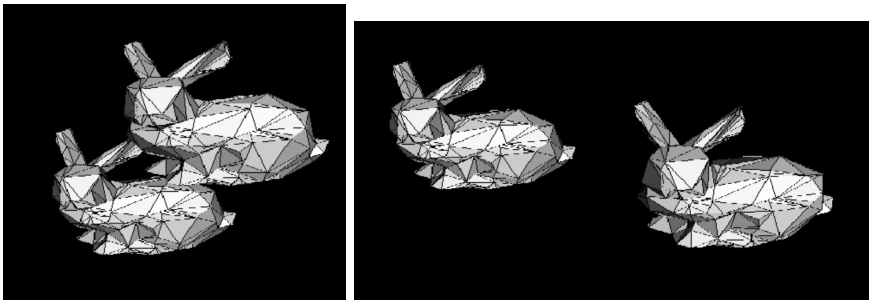

Figure 4: Physics simulation showing (left) collision between the falling bunny and the tracked bunny, and (right) the subsequent motion after collision as the bunny interacts with the floor.

5 Research

5.1 Dead Reckoning vs Accelerometer Fusion

Two approaches to orientation estimation were considered: gyroscope-only dead reckoning and a complementary filter combining gyroscope and accelerometer data. Dead reckoning updates orientation by integrating the measured angular velocity over time. This produces very smooth motion, but any gyroscope bias accumulates and causes drift. By contrast, accelerometer fusion uses gravity as a reference direction, allowing tilt drift to be corrected during playback.

To compare the two approaches, tilt error was computed by aligning the accelerometer measurements with the global frame and measuring the angular difference between the estimated gravity direction and the global up vector. From this, final, mean, RMS, and maximum tilt errors were recorded across the sequence, along with final absolute yaw and smoothness.

Method	Final Tilt	Mean Tilt	RMS Tilt	Max Tilt	Final |Yaw|	Smoothness
Gyro only	2.3961	3.0083	3.5743	32.0352	0.2135	0.0017
$\alpha = 0.01$	0.0065	0.0146	0.0435	3.2542	0.1379	0.0124
$\alpha = 0.30$	0.1321	0.4051	0.5500	5.6409	0.1550	0.0085
$\alpha = 0.70$	0.0232	0.9938	1.3652	17.2757	0.1922	0.0042
$\alpha = 0.98$	0.6400	1.7302	2.4098	28.8704	0.2102	0.0017

Table 3: Comparison of dead reckoning and accelerometer fusion using tilt-based error metrics.

The results show a clear trade-off between drift correction and smoothness. Gyroscope-only dead reckoning produced the smoothest motion, with a smoothness value of 0.0017, but also gave the largest tilt errors across the sequence, including a mean tilt error of 3.0083° and a maximum tilt error of 32.0352°. This confirms that dead reckoning is visually stable but prone to long-term tilt drift.

In contrast, accelerometer fusion substantially reduced tilt error for all tested values of α. The strongest correction occurred at $\alpha = 0.01$, which reduced the mean tilt error to 0.0146° and the maximum tilt error to 3.2542°. However, this came at the cost of much noisier motion, as reflected in the higher smoothness value of 0.0124. Increasing α reduced the aggressiveness of the correction and produced smoother motion, but also allowed more tilt drift to remain.

This demonstrates the main advantage of including the accelerometer: it provides a gravity reference that strongly limits tilt drift. The disadvantage is that aggressive correction can introduce visible instability, especially at low α values. In this implementation, higher values such as $\alpha = 0.98$ produced motion closer to dead reckoning in smoothness while still improving the accuracy of the tilt, while lower values produced the most accurate tilt estimates but the least smooth motion.

5.2 Computational Performance

The three tracking methods were also compared in terms of runtime. For the full IMU sequence, gyroscope-only dead reckoning required 0.2194s, gyroscope plus accelerometer fusion required 0.8857s, and gyroscope plus accelerometer plus magnetometer fusion required 1.8422s.

These results are consistent with the number of operations performed by each method. Dead reckoning is the cheapest because it only requires quaternion integration and normalisation. Adding accelerometer fusion increases the computational cost because each update also requires vector normalisation, rotation into the global frame, tilt-axis computation, and construction of a corrective quaternion. Including the magnetometer increases the cost further, since the magnetic field vector must also be rotated, projected onto the horizontal plane, compared with the reference direction, and used to form an additional yaw correction.

Although the fused methods are more expensive than dead reckoning, the absolute runtimes remain small. In practice, the additional computation was minor compared with the full rendering loop, so all three approaches were still suitable for real-time. This means the improved drift compensation from sensor fusion comes at a measurable but acceptable computational cost.

5.3 Positional Tracking Feasibility

In theory, positional tracking could be obtained from the IMU by integrating the accelerometer signal. After removing the gravity component, linear acceleration can be integrated once to obtain velocity and then integrated again to obtain position.

However, this approach is not reliable in practice. Accelerometer measurements contain both noise and bias, and these errors accumulate rapidly when integrated. Since position requires double integration, even very small errors grow quickly and lead to significant drift over time. In addition, accurate removal of gravity depends on a precise estimate of orientation, meaning any small orientation error also directly affects the computed acceleration and further increases positional drift.

In this coursework, the accelerometer was successfully used to correct tilt within the complementary filter, as this only requires short-term alignment with gravity. However, extending this to position estimation would require long-term stability, which is not achievable without an external reference. Unlike orientation, there is no natural correction mechanism for position in this setup, so errors accumulate without bound.

As a result, while orientation tracking using sensor fusion is effective, positional tracking using only IMU data is not feasible for sustained use. Reliable position estimation would require additional sensors or external references, such as visual tracking or GPS.

5.4 Collision Detection Using Spheres

Collision detection was implemented using bounding spheres, where a collision is detected when the distance between two object centres becomes smaller than the sum of their radii. The main advantage of this approach is efficiency: only a single distance check is required, so the cost per collision test is very low. This made it suitable for the real-time demonstration, especially given that the renderer itself was already computationally expensive.

The main disadvantage is that a sphere is only a rough approximation of the bunny mesh. The bunny model has an irregular shape with protruding regions such as the ears and a narrower body, so a single sphere does not fit it tightly. This can lead to inaccurate collision behaviour. For example, one bunny may visually collide with another bunny's ears, but this interaction may not be detected if the bounding spheres do not overlap. Similarly, collisions may also be detected too early when the spheres intersect but the visible meshes are still separate.

A more accurate alternative would be to use tighter bounding volumes such as axis-aligned bounding boxes (AABB), oriented bounding boxes (OBB), or multiple spheres attached to different parts of the bunny. These would better approximate the actual geometry and reduce false positives. A further improvement would be mesh-based collision detection, which would give the most accurate result by checking the actual triangles of the model.

However, these improvements would increase computational cost. AABB and OBB tests are more expensive than a single sphere test, and mesh-based collision detection would be significantly slower because many triangle-level intersection checks may be required. For the bunny used here, the sphere approximation gave a reasonable trade-off between simplicity and performance, but it sacrifices geometric accuracy.